MISSION CONTROL CENTER

The Complete Blueprint · v2 — "AI Agent Operating System"

Legacy Automations · AI Operating Systems

Owner: Lew (Founder / CEO) · Compiled June 9, 2026
Status: v2 BUILT & VERIFIED — 7 phase commits, full regression + headless boot pass

> *North star: when this loads, it should feel like the bridge of a working ship — not a website. Every glow, every moving avatar, every voice line is driven by a real event from the live backend. Futuristic comes from light + motion + coherence; trust comes from the fact that nothing on screen is fake.*

1 · Vision & Overview

Mission Control is a locally-running **AI agent operating system dashboard**: the operator's command center where AI employees are **visible** (live statuses, a game-like Living Office, real chat), **controllable** (approval gate, voice orders, a model-backed digital twin), and **accountable** (every number traceable to a real source or honestly labeled "not connected").

Two doctrines govern everything:

1. Honest data. A surface reports "live" only when its data path is implemented AND authorized. No fabricated numbers, no scripted chat, no fake "connected" badges. Fake data is a ship-blocking bug.

2. Approval before launch. Agents can request approvals but never resolve them. Outward-facing or irreversible actions always queue for the human operator.

v2 adds the experience layer on top of the verified v1 backend: a config-driven template (sellable to clients), the Command Deck visual system, the event-driven Living Office, the Legacy Lew digital twin, the pop-out Companion widget, and voice.

2 · Architecture

Zero-dependency Node backend (localhost-only) + React-via-Babel frontend + three agent control surfaces. Secrets live in the environment only and never reach the browser.

Layer	What it is	Key files
Frontend	React 18 via Babel-standalone; window.MC data layer; SSE live bridge; PixiJS scene	index.html, *.jsx, live.js, voice.js
Config layer	client.config.json loaded sync before app; brand/agents/twin/features applied at first paint	client.config.json, config-loader.js, config-apply.js
Backend	Zero-dependency Node HTTP server: static files + JSON API + SSE, file-backed persistent state	server.js, store.js, config.js
Vault	Reads the real Obsidian vault (Memory Galaxy/Pulse); guarded agent write path	server/vault.js
Integrations	Gmail / Calendar / Drive (one OAuth consent) + Notion (token) — all read-only, honest states	server/google.js, notion.js, oauth-google.js
Digital Twin	Anthropic Messages API persona with live ship context injected; key server-side only	server/twin.js, prompts/twin.md
Agent surfaces	MCP server (10 tools, stdio JSON-RPC), REST API, CLI — same live state	server/mcp.js, bin/mc

Event spine: every state change (chat, agent status, approval, vault write) is emitted once on an internal bus → persisted → fanned out over Server-Sent Events. The dashboard, the Living Office, the Companion widget and the voice layer are all just renderers of that one real event stream.

3 · v2 Feature Set

3.1 · Config-driven template layer (the client-template rule)

All client-specific state lives in **client.config.json** + **.env** + **assets/** — zero code edits per client. The config carries: brand (name, logo, accent colors, display font), operator identity, the agent roster (sprite, reserved 3D-model slot, desk slot, voice rate/pitch), twin settings (model, persona file, max tokens), and feature flags (livingOffice, voice, companion, integrations). A sync loader runs before all app scripts; missing config falls back to safe defaults.

3.2 · Command Deck reskin

Additive design layer (styles/command-deck.css): deck atmosphere (layered radial light + a slowly drifting circuit grid), frosted-glass panels (blur + saturation), one-shot edge-light sweeps on panel mount, and the governing rule **glow = status** — cyan live, gold needs-operator, crimson risk; if it glows, you can act on it. Respects prefers-reduced-motion. No layout rewrites: all v1 information density preserved.

3.3 · The Living Office (event-driven game scene)

The Paperclip/Teams page becomes a real-time 2.5D game scene (PixiJS) of the office: circuit floor, agent desks with accent-lit screens, the War Room table, the gold Approval Gate, the Vault archive wall, and an offline bench. The cast is the real brand chibi art (char-sage/kratos/faye/lew cutouts; honest branded chip for agents without art yet). Real game tech: eased walking with facing flips, idle/walk bob, dynamic shadows, y-depth sorting, HTML speech bubbles showing the actual message text.

Anti-gimmick principle: avatars move only because the system did something. The scene subscribes to the same SSE stream as the chat:

Real event	Scene behavior
Agent posts in War Room	Avatar walks to the table, speech bubble with the real text; desk-mates glance over
Operator DMs an agent	That agent walks to the command desk
Agent reply in a DM	Speech bubble at the agent
Status -> working	Walks to desk; desk screen glows in the agent's accent
Status -> offline	Walks to the bench, dims
Approval requested	Requester walks to the gold gate; beacon pulses until resolved
Approval resolved	Beacon flashes green (approve) / red (hold); requester returns
Vault note written	A glowing note card flies from the writer to the Vault wall

Honest states: LIVE vs DEMO HUD badge; if PixiJS or WebGL is unavailable the page falls back to the classic Teams view (which remains available on a toggle regardless).

3.4 · Legacy Lew — the Digital Twin

A model-backed persona endpoint: server/twin.js calls the Anthropic Messages API directly (plain fetch, zero SDK). The API key stays server-side, period. Every exchange injects **live ship context** — agent statuses, pending approvals, vault stats + keyword-matched notes, unread mail when Gmail is connected — so the twin answers "what needs me today?" from reality. Messaging Legacy Lew's direct channel in the dashboard triggers an automatic twin reply over SSE, like any agent. Model is one config line (TWIN_MODEL, default

claude-sonnet-4-6); spend is capped via TWIN_MAX_TOKENS. Persona doctrine (prompts/twin.md): never fabricate, never resolve approvals, keep it tight. No key -> honest offline notice, never canned fake replies.

3.5 · Companion widget (the twin outside the dashboard)

companion.html — a standalone pop-out window ("■ Companion" button by the LIVE badge): floating animated Lew avatar, **Today's Brief** (server-composed from real data: approvals, team, vault pulse, inbox, calendar — unconnected surfaces say so), an ask-the-twin input, a speak-replies toggle, and system notifications fired from the live SSE stream on gold-tier events (new approvals, system alerts). Upgrade path: the same page wraps into a Tauri menu-bar app (~3MB) with a global hotkey — runbook included.

3.6 · Voice

Tier 1 ships free with zero keys: push-to-talk mic buttons (Web Speech API) in the War Room and agent chat composers — push-to-talk by design, no always-on microphone — plus per-agent TTS voices (rate/pitch from config) and a persisted speak-replies toggle. Tier 2: the twin tries POST /api/tts (ElevenLabs proxy, server-side key) first and falls back honestly to the browser voice.

4 · Backend — API, MCP, CLI

4.1 · REST endpoints (localhost:8754)

Endpoint	Purpose
GET /api/health	Liveness check
GET /api/bootstrap	One snapshot: vault, agents, approvals, connections
GET /api/vault · GET/POST /api/vault/note	Galaxy/stats · read note · guarded agent WRITE (create/append/overwrite, vault-confined, .md o
GET/POST /api/chat	Channel history · post message (operator/agent roles)
GET /api/agents · POST /api/agents/update	Roster + live status · agent check-in
GET/POST /api/approvals · POST /api/approvals/resolve	Human gate: request / list / resolve (operator only)
GET /api/stream	Server-Sent Events: chat, agent, approval, vault events
GET /api/gmail | /api/calendar | /api/drive | /api/notion	Live integrations (read-only, cached, honest connected:false until authorized)
GET /api/twin/status · POST /api/twin/chat	Twin state · ask the twin (posts both sides to his channel)
GET /api/brief	Today's Brief — server-composed, every line from a real source
POST /api/tts	ElevenLabs premium-voice proxy (503 until keyed; browser voice fallback)

4.2 · MCP server (server/mcp.js) — 10 tools

mc_status, chat_read, chat_post, agent_set_status, vault_search, vault_read_note, **vault_write_note**, vault_recent, approval_request, approval_list. Agents can request approvals; resolving is operator-only. Register via mcp.config.example.json (stdio, MC_API env).

4.3 · CLI (bin/mc)

mc status · say · read · set · search · note · **write** · approvals · ask — wraps the same API for terminals, scripts, and cron.

5 · File / Folder Structure

```
legacy-mission-control/
■■ index.html · app.jsx · *.jsx (pages, components, data seeds)
■■ client.config.json · config-loader.js · config-apply.js (template layer)
■■ office.jsx (Living Office) · live.js (SSE bridge) · voice.js · companion.html
■■ styles/ (shell, pages, command-deck.css)
■■ assets/ (logos, char-*.png cutout sprites, avatar 3-angle set, circuit floor)
■■ prompts/twin.md (twin persona)
■■ server/ server.js · store.js · vault.js · config.js · google.js · notion.js
■         · oauth-google.js · twin.js · mcp.js · state/ (gitignored runtime + tokens)
■■ bin/mc (CLI) · package.json (start / mcp / google-auth)
■■ docs/ HANDOFF_GO_LIVE · GO_LIVE_CHECKLIST · DEBUG_REPORT · UPGRADE_BLUEPRINT
         · PRD_MISSION_CONTROL_TEMPLATE · BUILD_LOG_V2 · RUNBOOK_3D_VOICE_TAURI
```

6 · Setup & Run

6.1 · Start the ship

```
cd ~/Desktop/legacy-mission-control && npm start
→ http://127.0.0.1:8754/ (green LIVE badge = backend + vault connected)
```

Localhost-only by design. Never expose beyond 127.0.0.1 without adding an auth layer.

6.2 · Bring the twin online

```
export ANTHROPIC_API_KEY=sk-ant-... # usage-billed; separate from claude.ai subscription
npm start
```

6.3 · Google (Gmail + Calendar + Drive — ONE consent, all read-only)

```
1. console.cloud.google.com → project → enable Gmail API + Calendar API + Drive API
2. Credentials → OAuth client ID → type "Desktop app"
3. export GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=...
4. npm run google-auth # one-time consent; token saved gitignored, auto-refreshes
```

6.4 · Notion

```
notion.so/my-integrations → internal integration → export NOTION_TOKEN=ntn_...
Then in Notion: share the pages/databases with that integration.
```

6.5 · Environment knobs (.env.example)

Variable	Purpose	Default
MC_HOST / MC_PORT / MC_STATE_DIR	Bind + state location	127.0.0.1 / 8754 / server/state
VAULT_DIR	Obsidian vault path	~/Desktop/ai_2nd_Brain
GOOGLE_CLIENT_ID / SECRET	OAuth client (Desktop app)	— (required for Google three)
NOTION_TOKEN	Notion internal integration	—
ANTHROPIC_API_KEY	Digital Twin	— (twin honest-offline without it)
TWIN_MODEL / TWIN_MAX_TOKENS	Twin model + per-reply cap	claude-sonnet-4-6 / 1024
ELEVENLABS_API_KEY / ELEVEN_VOICE_ID	Premium twin voice	— (browser voice fallback)

7 · Verification Results (proof before claims)

Every check below was executed in an isolated test environment, not assumed:

Suite	Result
Syntax / parse: all server JS + 27 frontend files (Babel-verified)	PASS
v1 regression: REST, SSE (all 5 event types), CLI, MCP round-trip, persistence across restart, security guards (path traversal read+write+static, pay	82 PASS, 0 fail
Integrations: honest no-credential states; connector flip simulation; 14 mock-API shape tests (Gmail/Calendar/Drive/Notion → exact UI shapes)	PASS

Suite	Result
Twin mock suite: settings from config, live-context build, vault keyword hits, role mapping, request shape, never-fabricate rule present	11/11 PASS
Honest-offline: twin chat 503, TTS 503, brief "not connected" lines, single offline notice in channel	PASS
Full-page headless boot (jsdom + pinned React/Babel/Pixi against live backend): config loads, title branded, 18/18 PASS sprites merged, MCLive/MCVoice/Office	

Bugs found and fixed during self-recheck: Babel script-ordering bug (config/voice would have loaded before the data layer), a nonexistent icon, ungraceful WebGL failure in the office scene. Known limits: the Pixi scene's pixels can't be rendered in a headless sandbox — first visual pass happens on the operator's machine; twin/premium voice activate when their keys exist.

8 · Build Summary — the 7 v2 commits

Commit	Phase	Contents
6dcbf3d	Phase 0	Config-driven template: client.config.json + sync loader + applier (brand, operator, agents, twin, features)
8efc59f	Phase 1	Command Deck reskin: glass panels, deck atmosphere, glow-as-status, edge sweeps (additive, reduced-motion aware
9db3ee6	Phase 2	Living Office: PixiJS event-driven avatar scene + MCLive.onEvent fanout + companion pop-out button
83f7562	Phase 3	Digital Twin: server/twin.js + persona + live-context injection + auto-reply + /api/twin/* + /api/brief + /api/tts
982fb1f	Phase 4	Companion widget: companion.html with real Today's Brief + system notifications
6d59c3d	Phase 5	Voice: push-to-talk STT + per-agent TTS + speak toggle + premium-voice fallback chain
15d3b53	Phase 6	index.html wiring, runbooks (3D/voice/Tauri), .env template, BUILD_LOG_V2

Preceded by: initial dashboard · go-live backend (vault, chat/SSE, MCP/REST/CLI, de-fake) · vault writes · integrations go-live (Gmail/Calendar/Drive/Notion, debug sweep) · blueprint + PRD docs.

9 · Optional Upgrades (runbook'd, taste-gated)

True-3D avatars: the 3-angle Lew set + character cards are exactly what image-to-3D services (Meshy.ai, Tripo) need; download a rigged GLB into assets/models/ and set the agent's model3d config slot. Only swap when the GLB genuinely beats the sprite. **Menu-bar companion:** wrap companion.html in Tauri (~3MB tray app, global hotkey) — same page, zero rework. **Premium voice:** ELEVENLABS_API_KEY + voice ID; everything falls back honestly without it. **Template productization:** per the PRD — template repo → private client forks; Core / Living Office / Digital Twin tiers; 2-day deploy runbook; Legacy Automations itself is client #1.

Compiled from BUILD_LOG_V2.md, UPGRADE_BLUEPRINT.md, PRD_MISSION_CONTROL_TEMPLATE.md, DEBUG_REPORT.md and the v2 source tree. Doctrine throughout: honest data · approval before launch · proof before claims.